

Que la IA entienda tu codebase.

Cómo darle contexto de tu sistema, las reglas de negocio del banco y cómo validar lo que nos propone.

Lo que ya saben

Prompt engineering

Cómo pedir bien las cosas, roles, restricciones, criterio, etc.

Context engineering

Qué tiene sentido que entre y qué no en la ventana de contexto.

Skills

Prompts reutilizables por todo el equipo.

MCP

Conectar la IA a herramientas y datos.

Spec-Driven Development

Nuevo paradigma de programación basado en specs.

Validación

Tests, review y reglas de negocio antes de confiar.

Hasta acá vimos **cómo pedirle**. Hoy vamos a **darle con qué trabajar**.

El problema real

Reglas de negocio, permisos, flujos entre capas, integraciones, documentación interna, decisiones históricas y deuda técnica. Un core bancario es mucho más que el código que tenés abierto en el IDE.

La IA no ve nada de esto. **Por eso se equivoca.**

**El problema es que no te
avisa que no sabe algo,
lo inventa.**

Dos formas de usar IA

AUTOCOMPLETE

Completa la línea

No sabe de qué flujo es, ni qué reglas aplican, ni qué rompe. Te ahorra teclas.

ASISTENTE CONTEXTUALIZADO

Entiende el sistema

Sabe con qué trabaja, qué no puede tocar y cómo se chequea. Te ayuda a decidir.

¿Cómo le enseñamos a la IA nuestra codebase?

1 Archivos sueltos

Adjuntá al chat los archivos que importan. Copilot no siempre los cruza solo, los elegís vos.

2 Búsqueda textual

grep, @archivo o búsqueda del IDE. Encuentra coincidencias de texto, no cómo se conectan las piezas.

3 Docs y decisiones

Reglas de negocio y ADRs que el código no dice. Van en copilot-instructions, skills o specs del ticket.

4 RAG

Vectorizás docs y código en tu repo. Tu harness busca lo relevante por similitud semántica.

5 Grafo de conocimientos

Mapa de relaciones del repo, qué llama a qué, qué ruta expone qué servicio, sin leer archivo por archivo.

6 Memoria persistente (Engram)

Decisiones, convenciones y bugs resueltos quedan guardados. La IA los recupera en la próxima sesión, no arranca de cero.

Los embeddings encuentran **similitudes semánticas**. Los grafos, **relaciones determinísticas**. Y Engram **recuerda las**

¿Qué es un RAG?

Retrieval-Augmented Generation



Ejemplo: regla de transferencias

ANTES DE SUGERIR CÓDIGO, RECUPERAR

Tarea: modificar la validación de transferencias > 1M.

Recuperar primero:

- Doc de negocio del límite y excepciones
- Endpoints que llaman al validador
- Tests existentes del flujo
- Modelo de datos de la transacción
- Validadores previos y convenciones internas

Sin ese contexto, te tira algo que **parece bien** pero se saltea la excepción que pide la norma.

Qué darle de contexto y qué dejar afuera

✓ VA

Documentación de negocio y arquitectura

Contratos de API y OpenAPI

Tests, son ejemplos vivos de cómo se usa

Convenciones del equipo y ADRs

✗ NUNCA

Datos reales de clientes, nada de PII

Secrets, tokens y credenciales

Dumps y backups de producción

Logs con información sensible

¿Qué es un CAG?

Cache-Augmented Generation

CAG vs RAG

En tu repo ya lo hacés. Una parte va fija en cada sesión, la otra se activa cuando la tarea lo pide.

CAG

≈ **AGENTS.md**

Se inyecta siempre al arrancar la sesión.
Reglas del proyecto, convenciones,
contexto que no cambia en cada pregunta.

RAG

≈ **skills.md**

Se activa solo cuando la tarea lo necesita.
Busca por similitud en docs y código, no va
fijo en cada chat.

Se pueden combinar. **Ojo con apilar capas sin un propósito claro.**

Quién usa CAG y para qué

Sistemas que precargan lo que no cambia en cada consulta. Respuestas más rápidas, menos dependencia de búsqueda en vivo.

Banca e inversiones

Precarga normas de compliance y criterios de inversión. Los datos de mercado en vivo van por RAG.

Seguros

Precarga reglas de la póliza y flujos de siniestros. Aprueba casos simples sin buscar en cada consulta.

RRHH y reclutamiento

Precarga descripciones de puesto, criterios de hiring y normas laborales para matchear CVs al toque.

Soporte IT

Precarga runbooks, protocolos de troubleshooting y logs típicos para diagnosticar sin leer todo el repo.

Salud

Precarga guías clínicas y protocolos de tratamiento. El historial del paciente entra en tiempo real.

Tu repo con Copilot

copilot-instructions, ADRs, convenciones del equipo. Siempre en contexto, como un AGENTS.md.

¿Qué es un grafo de conocimientos?

Knowledge Graph

Grafo de conocimientos

Indexás el repo una vez y queda un mapa estructural. No es similitud textual: son vínculos explícitos entre piezas del sistema.



Grafo → impacto

Vectores → similitud

Grafo de conocimientos

Cómo representar relaciones reales dentro de una codebase y por qué reduce costo, latencia y trabajo innecesario.

Un grafo conecta las piezas de tu sistema



misma respuesta, una fracción del costo



• Costo

• Latencia

• Precisión

Grafo: pros y contras

✓ PROS

- Trazabilidad de impacto dentro del repo
- Relaciones explícitas, no suposiciones del modelo
- Menos ruido que explorar archivo por archivo

! CONTRAS

- △ Un grafo por repo, hexagonal distribuido necesita contratos aparte
- △ Mantenimiento: re-indexar cuando cambia el código
- △ Complejidad de armarlo y operarlo
- △ Falsa precisión si el grafo queda desactualizado

codebase-memory-mcp

Referencia open source del mix: grafo para impacto, semántica para lo que el nombre no dice. Un repo indexado, consultas vía MCP, 100% local.

QUÉ HACE

Indexa un repo en grafo persistente (tree-sitter + LSP)
14 herramientas MCP: trace, impact, search, architecture
El código no sale de tu máquina

CONSULTAS QUE HABILITA

trace → quién llama a esta función
impact → qué se rompe si cambio esto
search → encuentra por significado, no por nombre
architecture → mapa del módulo en un vistazo

QUICK START

```
# Instalar variante con UI (macOS / Linux)
curl -fsSL https://raw.githubusercontent.com/DeusData/codebase-memory-mcp/main/install.sh | bash -s --
--ui

# Levantar el grafo en el browser
codebase-memory-mcp --ui=true --port=9749
# → http://localhost:9749
```

¿Qué es Engram?

Memoria persistente entre sesiones

La IA que no arranca de cero

RAG, CAG y el grafo describen **tu sistema**. Engram recuerda **cómo trabajaste en él**: guarda lo que decidiste y lo recupera en la próxima sesión, aunque se reinicie el chat.

QUÉ GUARDA

- Decisiones y por qué se tomaron
- Convenciones del equipo y del proyecto
- Bugs resueltos y cómo se arreglaron
- Resúmenes de lo que se hizo en cada sesión

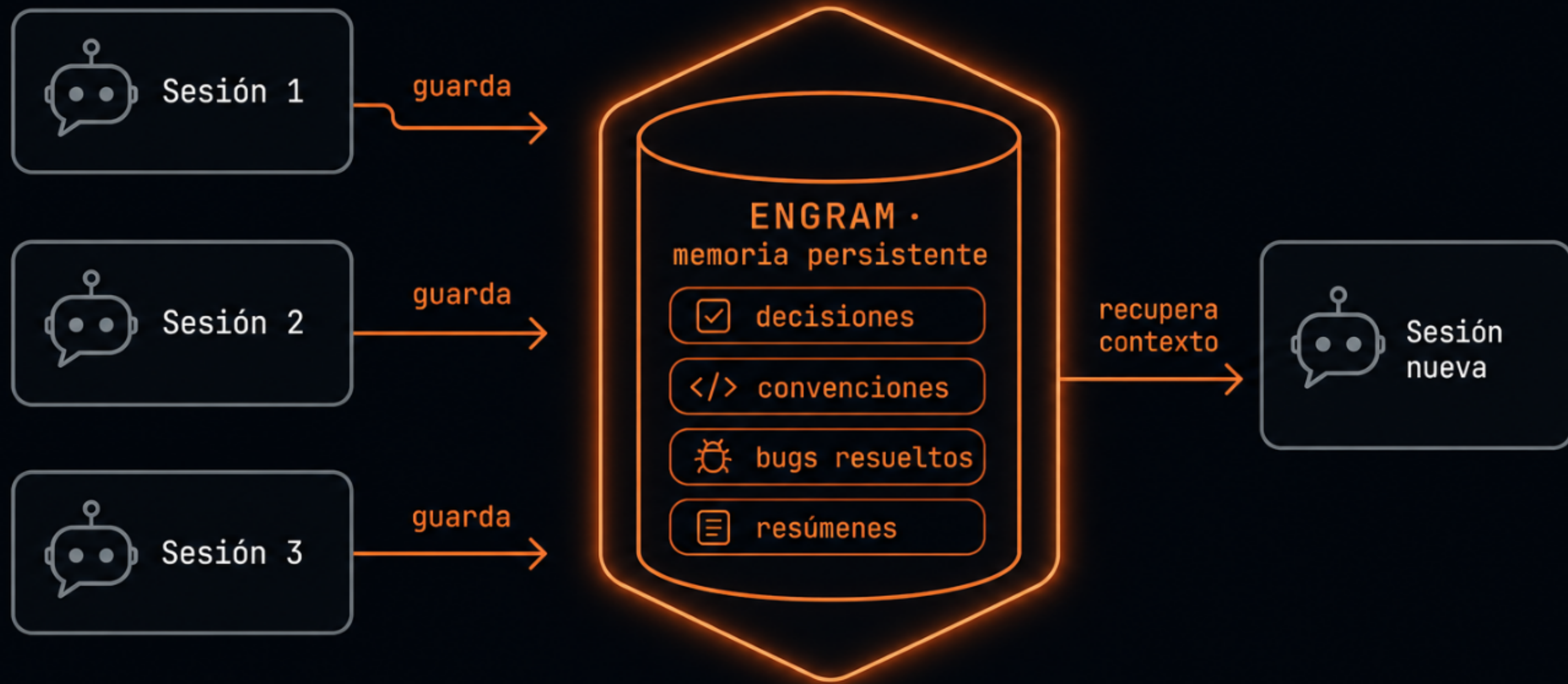
CÓMO ENCAJA EN EL BANCO

El conocimiento del equipo no se pierde al cerrar el chat

No repetís el mismo contexto en cada sesión

Corre local, vos decidís qué se guarda

Ojo: nada de PII ni secretos en la memoria



sobrevive entre sesiones y compactaciones

YA VIMOS CÓMO DARLE CONTEXTO

**La IA ya entiende tu sistema.
¿Cómo controlamos que trabaje
bien?**

Harness engineering

Todo lo que armás alrededor de la IA (contexto, reglas, herramientas, validaciones y límites) para que lo que produzca sea útil y se pueda verificar.

Qué compone un harness

LA BASE

Qué querés, con qué y bajo qué reglas.

- 01 Specs claras
- 02 Contexto correcto
- 03 Reglas del proyecto
- 04 Tests
- 05 Linters
- 06 CI/CD

EL CONTROL

Cómo se chequea antes de llegar a prod.

- 01 Validación de seguridad
- 02 Review humano
- 03 Checklists
- 04 Ambientes de prueba
- 05 Logs y trazabilidad
- 06 Límites explícitos

No es una herramienta, es **tener el control** de lo que produce la IA.

El harness en tu día a día

Cuatro tareas de todos los días. En cada una, los pasos para encararla con la IA sin romper nada, no un prompt suelto.

A Entender legacy

- Identificar archivos
- Pedir resumen + diagrama textual
- Marcar puntos de riesgo
- Validar contra el código real

B Cambiar regla de negocio

- Escribir la spec
- Recuperar contexto e impacto
- Proponer cambios, frenar ahí
- Generar tests + edge cases

C Tests sobre código existente

- Pedir matriz de casos
- Detectar ramas sin cubrir
- Generar y ejecutar
- Corregir con errores reales

D Refactor seguro

- Congelar comportamiento (tests antes)
- Cambios chicos
- Validación incremental
- Plan de rollback

Loops Engineering

Diseñas el ciclo, no cada prompt

Harness vs Loop

El harness es la base. El loop es el paso siguiente: el sistema vuelve solo, con tus reglas.

HARNESS

Vos abrís el chat

Reglas, specs, tests y verify

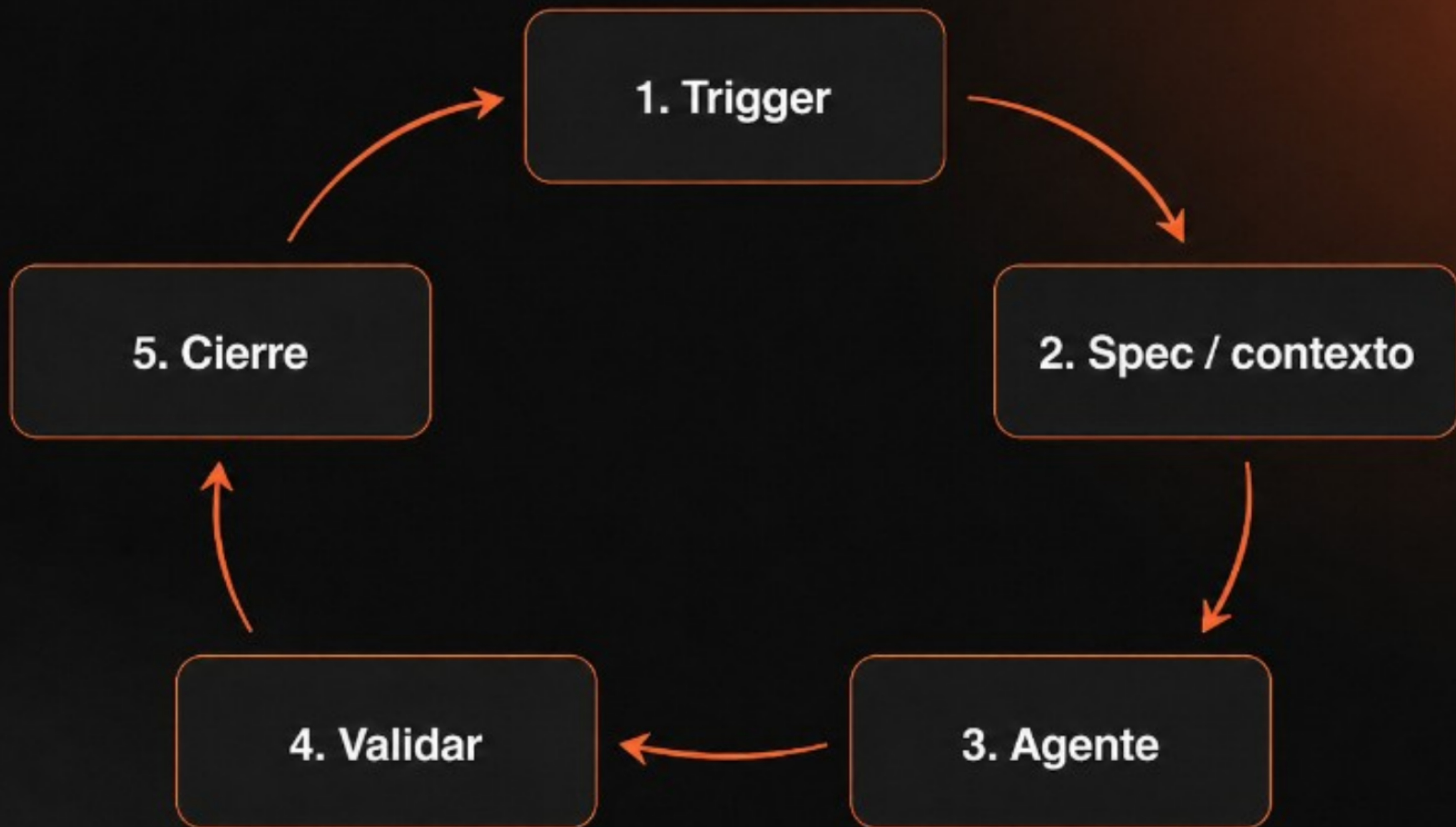
Un repo, un ticket, un cambio

LOOP

Corre solo (trigger o cron)

Se repite hasta cumplir criterio

Aprende del diff y del estado en repo



ticket → spec en repo → agente → npm test + review → merge

De qué está hecho un loop

No es solo el modelo iterando. Es trigger + herramientas + reglas + memoria en el repo.

Automations

Cron, webhooks, triggers

Skills

Procesos repetibles

MCP

Herramientas externas

Subagentes

Roles especializados

Markdown en repo

Estado que sobrevive

Verify

Tests y gates antes del cierre

Advertencias

Verificación tuya

El loop no reemplaza tu juicio

Deuda de comprensión

Si no entendés el diff, pará

Loop sin harness

Autonomía sin reglas = caos
automático

Construí el loop. **Seguí siendo el ingeniero.**

**El salto no es
una IA más potente.
Es mejor contexto, límites y
proceso.**

Para llevarse

Prompt engineering es la base

Context engineering es estratégico

RAG, grafos y memoria sirven como herramientas del context engineering

Harness engineering vuelve el flujo confiable

Loop engineering persigue la autonomía controlada en los agentes